数值计算与计算机应用
*Journal on Numerical Methods and Computer Applications*
ISSN 1000-3266,CN 11-2124/TP

# 《数值计算与计算机应用》网络首发论文

题目： pyRFM:基于 Python 的随机特征方法高性能实现与应用
作者： 陈景润，孙羿斐
收稿日期： 2025 年 8 月 31 日
网络首发日期： 2025-12-09
引用格式： 陈景润，孙羿斐．pyRFM:基于 Python 的随机特征方法高性能实现与应用[J/OL]．数值计算与计算机应用.
https://link.cnki.net/urlid/11.2124.TP.20251208.0935.004

**网络首发**：在编辑部工作流程中，稿件从录用到出版要经历录用定稿、排版定稿、整期汇编定稿等阶段。录用定稿指内容已经确定，且通过同行评议、主编终审同意刊用的稿件。排版定稿指录用定稿按照期刊特定版式（包括网络呈现版式）排版后的稿件，可暂不确定出版年、卷、期和页码。整期汇编定稿指出版年、卷、期、页码均已确定的印刷或数字出版的整期汇编稿件。录用定稿网络首发稿件内容必须符合《出版管理条例》和《期刊出版管理规定》的有关规定；学术研究成果具有创新性、科学性和先进性，符合编辑部对刊文的录用要求，不存在学术不端行为及其他侵权行为；稿件内容应基本符合国家有关书刊编辑、出版的技术标准，正确使用和统一规范语言文字、符号、数字、外文字母、法定计量单位及地图标注等。为确保录用定稿网络首发的严肃性，录用定稿一经发布，不得修改论文题目、作者、机构名称和学术内容，只可基于编辑规范进行少量文字的修改。

**出版确认**：纸质期刊编辑部通过与《中国学术期刊（光盘版）》电子杂志社有限公司签约，在《中国学术期刊（网络版）》出版传播平台上创办与纸质期刊内容一致的网络版，以单篇或整期出版形式，在印刷出版之前刊发论文的录用定稿、排版定稿、整期汇编定稿。因为《中国学术期刊（网络版）》是国家新闻出版广电总局批准的网络连续型出版物（ISSN 2096-4188，CN 11-6037/Z），所以签约期刊的网络版上网络首发论文视为正式出版。

网络首发时间：2025-12-09 14:23:56　网络首发地址：https://link.cnki.net/urlid/11.2124.TP.20251208.0935.004

2026 年 6 月　数值计算与计算机应用　第 47 卷第 2 期
Jun., 2026　Journal on Numerical Methods and Computer Applications　Vol.47, No.2

# pyRFM: 基于 Python 的随机特征方法高性能实现与应用*1)

陈景润
(中国科学技术大学, 中国科学技术大学苏州高等研究院, 苏州 215217)

孙羿斐2)
(苏州大学, 苏州 215006)

**摘　要**

随机特征方法 (Random Feature Method, RFM) 是一种新兴的偏微分方程求解框架, 它通过浅层神经网络构造逼近解的函数空间, 兼具传统数值方法的严谨性与现代机器学习的灵活性. 本文基于 Python 语言设计并实现了一个高性能的 RFM 求解工具 pyRFM. 在框架设计上, pyRFM 完整覆盖了几何表示与采样、特征矩阵组装与方程求解、结果可视化等关键环节, 为用户提供了从建模到数值实验的系统化流程. 与此同时, 软件在实现过程中充分结合了 RFM 方法的特性, 依托广泛使用且支持充分的 Python 科学计算库, 具备良好的易用性与可扩展性; 此外, pyRFM 在整个计算流程中均不依赖传统网格化操作, 从而在处理复杂几何时展现出更高的灵活性与效率.

**关键词:** 随机特征方法; 偏微分方程; 符号距离函数; Python; 数值求解.

**MR (2010) 主题分类:** 65K10, 90C30.

**DOI:** 10.12288/szjs.j2025-1045

**CSTR:** 32034.14.szjs.j2025-1045

# pyRFM: A HIGH-PERFORMANCE PYTHON IMPLEMENTATION OF THE RANDOM FEATURE METHOD WITH APPLICATIONS

Chen Jingrun
(*School of Mathematical Sciences and Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou 215217, China*)

Sun Yifei
(*School of Mathematical Sciences, Soochow University, Suzhou 215006, China*)

**Abstract**

The Random Feature Method (RFM) is an emerging framework for solving partial differential equations (PDEs). It constructs function spaces for approximating solutions through shallow neural networks, combining the rigor of traditional numerical methods with the flexibility of modern machine learning. In this work, we design and implement a high-performance

* 2025 年 8 月 31 日收到.
1) 基金项目: 国家自然科学基金 (92370205 和 12425113) 资助.
2) 通信作者: 孙羿斐, Email: yfsun99@stu.suda.edu.cn.

RFM solver, pyRFM, based on the Python programming language. In terms of framework design, pyRFM provides a complete workflow that covers key components including geometry representation and sampling, feature matrix assembly and equation solving, as well as result visualization, thereby enabling a systematic process from modeling to numerical experiments. Meanwhile, the software is developed with careful consideration of the intrinsic characteristics of RFM, leveraging widely used and well-supported Python scientific computing libraries to ensure ease of use and extensibility. Moreover, pyRFM is fully compatible with both CPU and GPU computing environments, and its entire workflow is free from traditional mesh-based operations, which grants it higher flexibility and efficiency when handling complex geometries.

**Keywords:** Random Feature Method; Partial Differential Equations; Signed Distance Functions; Python; Numerical Methods.

**2010 Mathematics Subject Classification:** 65K10, 90C30.

## 1. 引 言

在科学计算领域, 求解偏微分方程 (PDE) 的算法是最早被研究、也最为深入的课题之一. 有限差分法 [1]、有限元法 [2]、谱方法 [3] 以及众多其他方法已经被提出并取得了巨大成功. 与此同时, 基于这些方法的各类科学计算软件也相继开发出来, 并被学术界和工业界广泛应用, 几乎成为所有工程应用中的标准工具.

近年来, 随着神经网络模型在各类人工智能 (AI) 任务中取得显著突破, 将其应用于偏微分方程 (PDE) 求解的研究也迅速兴起 [4–9]. 深度神经网络具有强大的函数表达能力, 以物理信息神经网络 (Physics-Informed Neural Networks, PINNs) [8] 为代表的一系列方法通过在损失函数中引入物理约束, 使模型能够在高维空间中逼近复杂解结构, 并在反问题与不适定问题的处理上展现出独特优势. 与此类直接用神经网络逼近 PDE 解的方法不同, 另一条研究路径聚焦于学习输入函数空间到输出函数空间的映射, 即所谓的算子学习. 代表性成果包括深度算子网络 (DeepONet) [10] 和傅里叶神经算子 (Fourier Neural Operator, FNO) [11], 它们能够在一次训练后泛化到不同边界条件、源项或参数的 PDE 问题, 为 PDE 求解提供了更高效且具有普适性的框架.

深层模型往往伴随着训练困难、对超参数高度敏感以及计算代价较高等问题. 为此, 多项研究尝试采用浅层模型以提升训练效率. 其中, 极限学习机 (Extreme Learning Machine, ELM) [12] 提出使用单隐层前馈神经网络, 并强调即便在隐含层权重随机固定的情况下, 仍能逼近复杂函数. 有关随机特征 (Random Features, RF) [13,14] 的概念则起源于对核函数的近似研究, 其核心思想是通过构造低维、可显式计算的随机特征来逼近核函数, 在高维空间中实现高效的非线性建模 [14–18]. 两者在实现机制上具有一定的相似性. 在偏微分方程 (PDE) 问题上, 已有研究尝试利用浅层网络来结合传统数值方法与机器学习方法的优势 [19–24]. 例如, 局部极限学习机 (Local Extreme Learning Machine, LocEM) [21] 融合了 ELM、区域分解与局部神经网络的思想, 并在子域边界上引入 $C^k$ 连续性条件. 随机特征方法 (Random Feature Method, RFM) [22,25,26] 则通过设计具备 $C^k$ 连续性的单位分解函数, 将局部特征自然延拓至全局, 从而避免了子域边界采样. 基于随机神经网络与 Petrov-Galerkin 方法的 RNN-PG 框架 [24], 能够灵活选取测试函数 (有限元基、正交多项式或神经网络), 还在高维热方程与非线性 Burger 方程等问题上展现出高精度与高效率. 可迁移神经网络 (Transferable Neural Networks,

TransNet) [23] 通过引入预训练机制, 进一步增强了隐藏层特征的逼近能力. 在计算性能方面, 这类方法在精度与效率上往往可与有限元方法相媲美, 甚至更优 [21,22]. 同时, 作为机器学习方法, 随机特征方法继承了 PINN 的无网格优势, 相比经典有限元方法, 在某些网格化困难的复杂几何上展现出更强的灵活性与适应性 [22,25,26].

本文重点探讨随机特征方法 (RFM) 的具体实现, 其若干操作方式既区别于传统数值算法, 也不同于深度学习模型. 在实践中, RFM 常被视作深度学习的一种变体, 因而可借助主流深度学习框架实现高效的 GPU 并行计算. 然而, 在随机特征矩阵的组装及优化求解方面仍存在显著的改进空间. 例如, 由于网络结构相对简单, 矩阵组装过程往往无需依赖自动微分工具; 同时, 相较于基于随机梯度下降的优化策略, 采用线性或非线性最小二乘等数值代数技术, 往往能够更高效地求解随机特征方法中的优化问题 [27].

基于上述背景, 本文将系统梳理随机特征方法的实现路径, 并在此基础上开发出一个高性能、可扩展的实现框架 —— pyRFM. 该框架基于 Python 实现, 可通过 pip 直接安装, 并已开源于 https://github.com/IFaay/pyRFM, 以便学术研究与工程应用.

本文后续内容安排如下: 第 2 节将详细介绍随机特征方法; 第 3 至第 6 节分别探讨随机特征方法的实现细节, 包括几何表示、单位分解、随机特征矩阵的组装以及优化器的设计; 第 7 节通过一些案例展示了在软件框架下实现这些方法的简便性; 最后, 第 8 节对本文进行总结并展望软件未来的开发方向.

## 2. 随机特征方法

在随机特征方法 (Random Feature Method, RFM) 中, 特征函数源自两层神经网络, 其中内层参数被固定并随机选择. 具体而言, 设 $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{d\times N}$ 表示输入的 $d$ 维向量集合, $\boldsymbol{A} = [a_{i,j}] \in \mathbb{R}^{d\times M}$ 与 $\boldsymbol{b} = [b_j] \in \mathbb{R}^M$ 分别为随机生成的权重矩阵和偏置向量. 在给定随机权重 $\boldsymbol{A}$ 和偏置 $\boldsymbol{b}$ 的条件下, 可自然地构造一族多变量随机特征函数:

$$\phi(\boldsymbol{x})_j := \sigma\left(\sum_{k=1}^{d} a_{k,j}x_k + b_j\right), \tag{2.1}$$

其中 $\boldsymbol{x} = (x_k) \in \mathbb{R}^d$, $\sigma : \mathbb{R} \to \mathbb{R}$ 是逐元素作用的光滑激活函数. 在处理含时间变量的问题时, 我们引入时间变量 $t$ 及其对应的随机权重 $a_{d+1,j}(j = 1, \ldots, M)$. 此时, 时间相关的随机特征函数可采用以下两种形式之一:

$$\phi(\boldsymbol{x}, t)_j = \sigma\left(\sum_{k=1}^{d} a_{k,j}x_k + b_j\right) \cdot \sigma\left(a_{d+1,j}t + b_j\right)$$

或

$$\phi(\boldsymbol{x}, t)_j = \sigma\left(\sum_{k=1}^{d} a_{k,j}x_k + a_{d+1,j}t + b_j\right).$$

上述两种构造方式在文献中已有系统比较与分析 [28]. 随机特征函数的导数可通过链式法则计算.

考虑如下形式的偏微分方程问题:

$$\begin{cases} \mathcal{L}\boldsymbol{u}(\boldsymbol{x}) + \mathcal{N}\left(\boldsymbol{u}(\boldsymbol{x})\right) = \boldsymbol{f}(\boldsymbol{x}), & \boldsymbol{x} \in \Omega, \\ \mathcal{B}\boldsymbol{u}(\boldsymbol{x}) = \boldsymbol{g}(\boldsymbol{x}), & \boldsymbol{x} \in \partial\Omega, \end{cases} \tag{2.2}$$

其中 $\mathcal{L}$ 与 $\mathcal{N}$ 分别是线性与非线性的微分算子, $\Omega$ 为 $\mathbb{R}^d$ 中有界连通区域. 典型例子包括泊松方程、线性弹性问题及斯托克斯流动问题等.

为适应解的局部变化, 我们引入局部随机特征, 并通过单元覆盖的方式将其拼接在一起. 具体地, 从区域内选择一组中心点 $\{\boldsymbol{x}_n\}_{n=1}^{M_p} \subset \Omega$, 构造归一化坐标:

$$\tilde{\boldsymbol{x}} = \frac{1}{\boldsymbol{r}_n}(\boldsymbol{x} - \boldsymbol{x}_n), \quad n = 1, \ldots, M_p, \tag{2.3}$$

其中 $\boldsymbol{r}_n \in \mathbb{R}^d$ 是预设的尺度参数.

在 $d = 1$ 的情况下, PoU 函数的一个简单选择为指示函数:

$$\psi_n^a(x) = \mathbb{I}_{-1 \le \tilde{x} < 1}, \tag{2.4}$$

或连续可微函数:

$$\psi_n^b(x) = \begin{cases} \dfrac{1 + \sin(2\pi\tilde{x})}{2}, & -\dfrac{5}{4} \le \tilde{x} < -\dfrac{3}{4}, \\ 1, & -\dfrac{3}{4} \le \tilde{x} < \dfrac{3}{4}, \\ \dfrac{1 - \sin(2\pi\tilde{x})}{2}, & \dfrac{3}{4} \le \tilde{x} < \dfrac{5}{4}, \\ 0, & \text{其他情况}. \end{cases} \tag{2.5}$$

高维 PoU 函数可通过一维函数的张量积构造: $\psi_n(\boldsymbol{x}) = \prod_{k=1}^d \psi_n(x_k)$. 对于复杂的几何区域 $\Omega$, 可以根据其包围盒进行子域的划分, 从而确定定义单位分解的中心点 $\boldsymbol{x}_n$ 和尺度参数 $\boldsymbol{r}_n$, 子域划分的细节将在第 4 节进行具体说明.

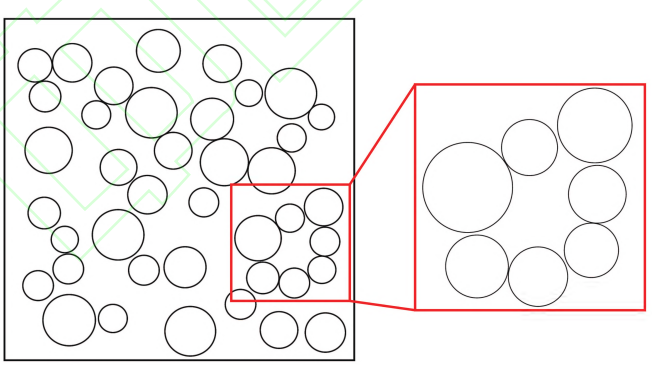


图 1 复杂几何示例

因此, 最终的近似解为:

$$u_M(\boldsymbol{x}) = \sum_{n=1}^{M_p} \psi_n(\boldsymbol{x}) \sum_{j=1}^{M_n} u_{nj} \phi_{nj}(\boldsymbol{x}). \tag{2.6}$$

针对问题 (2.2), 一个简单的损失函数可定义为:

$$\text{Loss} = \sum_{\boldsymbol{x}_i \in \Omega} \lambda_{Ii} \left\| \mathcal{L}\boldsymbol{u}_M(\boldsymbol{x}_i) + \mathcal{N}\boldsymbol{u}_M(\boldsymbol{x}_i) - \boldsymbol{f}(\boldsymbol{x}_i) \right\|_{l^2}^2 + \sum_{\boldsymbol{x}_k \in \partial\Omega} \lambda_{Bk} \left\| \mathcal{B}\boldsymbol{u}_M(\boldsymbol{x}_k) - \boldsymbol{g}(\boldsymbol{x}_k) \right\|_{l^2}^2, \tag{2.7}$$

其中 $\{\lambda_{Ii}\}$ 与 $\{\lambda_{Bk}\}$ 是惩罚系数, $\{\boldsymbol{x}_i\} \subset \Omega$ 与 $\{\boldsymbol{x}_k\} \subset \partial\Omega$ 分别为内部与边界上的配置点. 当方程中仅包含线性算子时, 损失函数 (2.7) 的最小化问题为凸优化问题, 可等价地转化为线性最小二乘问题求解. 当方程中包含非线性算子时, 可以考虑通过牛顿法等方法进行线性后, 转化为线性最小二乘问题迭代求解. RFM 在传统方法与机器学习方法之间架起桥梁, 在精度和计算效率上可媲美传统数值求解器, 同时继承了机器学习算法在处理复杂几何结构方面的优势, 例如处理一些网格化困难的窄缝, 如图 1.

## 3. 几何表示

几何的表达方式多种多样, 通常可分为显式表示与隐式表示两类. 显式表示 (Explicit Representation) 是通过参数化方式直接描述几何对象的位置、形状或边界, 常见的形式包括网格、点云、B 样条曲面以及 NURBS 曲面等. 在传统数值计算方法中, 例如有限元方法, 通常依赖于显式网格的生成, 而网格质量对计算精度有显著影响. 当面对复杂几何形状难以生成高质量网格时, 此类方法的适用性将受到限制.

相较之下, 隐式表示 (Implicit Representation) 通过一个函数来定义几何体, 其形状由该函数为零的点集所确定. 本文采用的是一种典型的隐式表达方式 —— 符号距离函数 (Sig- ned Distance Function, SDF) [29]. 该方法在构造复杂几何时更为灵活, 为后续研究提供了便利.

**定义 3.1 (符号距离函数).** 对于 $n$ 维空间中的一个集合 $\Sigma \subset \mathbb{R}^n$, $\boldsymbol{x} \in \mathbb{R}^n$ 是任意点, 可定义距离函数

$$\mathrm{dist}(\boldsymbol{x}, \Sigma) = \min_{\boldsymbol{y} \in \Sigma} \|\boldsymbol{x} - \boldsymbol{y}\|,$$

其中 $\|\cdot\|$ 表示欧几里得范数.

对于几何体 $\Omega$, $\partial\Omega$ 是几何体的边界, 可以定义表达目标几何的符号距离函数 $\phi$

$$\phi(\boldsymbol{x}) = \begin{cases} -\mathrm{dist}(\boldsymbol{x}, \partial\Omega), & \boldsymbol{x} \in \Omega \text{ (内部)} \\ \mathrm{dist}(\boldsymbol{x}, \partial\Omega), & \boldsymbol{x} \notin \bar{\Omega} \text{ (外部)} \\ 0, & \boldsymbol{x} \in \partial\Omega \end{cases}$$

其中 $\mathrm{dist}(\boldsymbol{x}, \partial\Omega)$ 表示点 $\boldsymbol{x}$ 到几何体边界 $\partial\Omega$ 的最小欧氏距离.

软件基于一个统一的模板, 定义了一系列常用的几何体, 线段, 矩形, 圆, 长方体, 球体, $n$ 维超立方体等, 这些几何体均继承自一个几何基类 1. 该基类定义了几何体的维度、符号距离函数、包围盒以及采样方法等基本属性和方法. 通过继承该基类, 可以方便地扩展新的几何体类型.

```python
class GeometryBase(ABC):
    # 定义几何体的维度与本质维度
    def __init__(self, dim=None, intrinsic_dim=None):
        self.dim = ...
        self.intrinsic_dim = ...
        self.boundary = []

    # 定义符号距离函数
    @abstractmethod
```

```python
    def sdf(self, p: torch.Tensor):
        pass

    # 定义包围盒
    @abstractmethod
    def get_bounding_box(self):
        pass

    # 定义几何体内部点采样方法
    @abstractmethod
    def in_sample(self, num_samples, with_boundary=False):
        pass

    # 定义几何体边界点采样方法
    @abstractmethod
    def on_sample(self, num_samples, with_normal=False):
        pass

    # 几何交集
    def __and__(self, other):
        return IntersectionGeometry(self, other)

    # 几何并集
    def __or__(self, other):
        return UnionGeometry(self, other)

    # 几何补集
    def __invert__(self):
        return ComplementGeometry(self)
```

Listing 1 几何基类

使用符号距离函数 (Signed Distance Function, SDF) 作为几何表示的优势在于, 可以通过简单的数学运算实现几何体之间的交集、并集和差集操作, 从而以基础几何构建出复杂的几何形状. 对于两个由 SDF 定义的几何体 $\phi_1$ 和 $\phi_2$, 其交集可表示为 $\phi_\cap = \min\{\phi_1, \phi_2\}$, 并集为 $\phi_\cup = \max\{\phi_1, \phi_2\}$, 补集为 $\phi_- = -\phi$. 差集则可定义为一个几何体与另一个几何体补集的交集. 对于经过布尔运算得到的几何, 可以递归地采样构造其的几何体, 再利用 SDF 对采样点进行筛查, 可以方便的实现复杂几何体的采样. 如图 1 所示的几何体, 可由一个正方形与若干圆形的差集得到.

## 4. 单位分解

通过将求解区域划分为多个子区域, 并在每个子区域上进行对应的尺度调整, 可以构造局部随机特征函数. 子区域之间的连接需要通过添加与方程阶数相关的连续性条件来实现. 此外, 还可以构造满足相应阶数连续性条件的单位分解函数, 将局部随机特征函数扩展到整个求解区域. 区域分解在提升随机特征函数的逼近能力方面具有显著效果.

```python
class POUBase(ABC):
    # 抽象基类，表示单位分解(Partition of Unity, POU)函数的接口

    @abstractmethod
    def forward(self, x):
        # 前向函数： 计算单位函数在点x 上的函数值
        pass

    @abstractmethod
    def first_derivative(self, x, axis):
        # 一阶导数： 计算单位函数在点x 上沿指定轴的偏导数
        pass

    @abstractmethod
    def second_derivative(self, x, axis1, axis2):
        # 二阶导数： 计算单位函数在点x 上沿指定轴1和轴2的二阶偏导数
        pass

    @abstractmethod
    def higher_order_derivative(self, x, order):
        # 高阶导数： 计算单位函数在点x 上的指定阶导数(order 是一个多重索引)
        pass
```

Listing 2 单位分解函数基类

本文提供两种通用的子区域的划分方法: 一是, 根据几何体的包围盒 (bounding box) 进行 (每个维度上的) 等距划分. 二是, 结合 K-Means 聚类结果, 使用 Voronoi 图对几何体进行划分.

对于划分方法一, 我们首先计算几何体的包围盒 (bounding box), 然后在每个维度上将包围盒等距划分为若干个区间, 从而形成规则的子区域. 具体步骤如下:

1. 计算几何体的包围盒, 确定每个维度上的最小值 $\mathrm{lb}_i$ 和最大值 $\mathrm{ub}_i$, 其中 $i = 1, \ldots, d$ 表示维度.

2. 在每个维度上将区间 $[\mathrm{lb}_i, \mathrm{ub}_i]$ 等距划分为 $n_i$ 份, 划分的数量 $n_i$ 可由用户指定.

3. 通过笛卡尔积生成所有子包围盒的边界, 子包围盒的中心点可表示为:

$$c_{j_1,\ldots,j_d} = \left(\mathrm{lb}_1 + \frac{(2j_1-1)(\mathrm{ub}_1-\mathrm{lb}_1)}{2n_1}, \ldots, \mathrm{lb}_d + \frac{(2j_d-1)(\mathrm{ub}_d-\mathrm{lb}_d)}{2n_d}\right),$$

   其中 $j_i = 1, \ldots, n_i$.

4. 每个子包围盒的中心点 $c_{j_1,\ldots,j_d}$ 和其边界 $[\mathrm{lb}_i + \frac{(j_i-1)(\mathrm{ub}_i-\mathrm{lb}_i)}{n_i}, \mathrm{lb}_i + \frac{j_i(\mathrm{ub}_i-\mathrm{lb}_i)}{n_i}]$ 定义了子包围盒. 子区域是几何体与子包围盒的交集.

对于划分方法二, 我们结合 K-Means 聚类算法和 Voronoi 图对几何体进行划分. 具体步骤如下:

1. 在几何体内部均匀采样若干点 $\{\boldsymbol{x}_i\}_{i=1}^N$, 这些点将作为 K-Means 聚类的输入数据.

2. 使用 K-Means 算法对采样点进行聚类, 得到 $K$ 个聚类中心 $\{\boldsymbol{c}_k\}_{k=1}^K$, 每个聚类中心对应一个子区域.

3. 基于聚类中心构造 Voronoi 图. Voronoi 图将整个几何体划分为若干个子区域, 每个子区域包含距离某个聚类中心最近的所有点.

4. 对于每个 Voronoi 子区域, 进一步取其与几何体的交集, 得到最终的子区域划分结果.

方法二的优势在于能够根据几何体的形状和采样点的分布自适应地划分子区域, 从而更好地适应复杂几何结构和局部特征的变化.

单位分解函数 (Partition of Unity, PoU) 需要满足的一个基本要求是: 在整个求解区域内, 所有局部函数的和为 1. 具体地, 对于任意点 $\boldsymbol{x} \in \Omega$, 有

$$\sum_{n=1}^{M_p} \psi_n(\boldsymbol{x}) = 1,$$

其中 $\psi_n(\boldsymbol{x})$ 是第 $n$ 个局部单位分解函数, $M_p$ 是子区域的数量. 为了满足这一要求, 本文采用了一种先取值再归一化的策略, 而非在构造函数时进行复杂的分类讨论. 这种方法的优势在于避免了在构造函数时进行复杂的分类讨论, 同时确保了单位分解函数的和为 1. 此外, 这种归一化策略能够适应非均匀的分割方式, 适用于各种几何体的划分.

## 5. 随机特征矩阵的组装

随机特征矩阵 定义为 $\boldsymbol{\Phi} = [\phi_{i,j}] := [\sigma(\langle \boldsymbol{x}_i, \boldsymbol{a}_j \rangle + b_j)] \in \mathbb{R}^{N\times M}$, 其中内积定义为 $\langle \boldsymbol{x}_i, \boldsymbol{a}_j \rangle := \sum_k x_{i,k} a_{k,j}$. 也可将其重写为 $\boldsymbol{\Phi} = \sigma(\boldsymbol{X}^* \boldsymbol{A} + \boldsymbol{b})$, 其中激活函数逐元素作用, 偏置向量逐列加上. 矩阵中的列向量 $\{\boldsymbol{\phi}_j\}$ 即为随机特征. 设 $\boldsymbol{\Phi}^*$ 表示 $\boldsymbol{\Phi}$ 的共轭转置. 当 $\boldsymbol{\Phi} \in \mathbb{R}^{M\times N}$ 满秩时, 其伪逆为:

$$\boldsymbol{\Phi}^\dagger = \begin{cases} \boldsymbol{\Phi}^*(\boldsymbol{\Phi}\boldsymbol{\Phi}^*)^{-1}, & 若 M \le N, \\ (\boldsymbol{\Phi}^*\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^*, & 若 N \ge M. \end{cases}$$

矩阵 $\boldsymbol{\Phi}$ 相对于 2- 范数的条件数定义为 $\kappa(\boldsymbol{\Phi}) := \|\boldsymbol{\Phi}\|_2 \|\boldsymbol{\Phi}^\dagger\|_2$. 记 $\boldsymbol{\Phi}$ 的第 $k$ 个奇异值为 $\lambda_k(\boldsymbol{\Phi})$. 对于偏微分方程 (PDE), 我们构造如下形式的近似解:

$$u_M(\boldsymbol{x}_i) = \sum_{j=1}^{M} u_j \phi_{i,j}(\boldsymbol{x}_i),$$

即 $\boldsymbol{u}_M(\boldsymbol{X}) = \boldsymbol{u}^* \sigma(\boldsymbol{X}^* \boldsymbol{A} + \boldsymbol{b}) = \boldsymbol{u}^* \boldsymbol{\Phi} \in \mathbb{R}^{d_{\text{out}}\times N}$, 其中 $\boldsymbol{u} \in \mathbb{R}^{M\times d_{\text{out}}}$ 为待求系数.

随机特征矩阵是随机特征函数经微分算子作用后, 在配点处的取值. 在随机特征方法 (RFM) 中, 通常采用单隐藏层神经网络作为随机特征函数, 其导数及高阶导数往往具有统一的表达形式或可递推的计算公式. 与机器学习方法中常用的自动微分相比, 使用明确形式的导函数显著减轻了内存负担. 在软件中, 预先设置了几类常用的激活函数以及对应的导函数的显式表达. 对于更为一般形式的激活函数, 我们提供了一个模板, 可以基于随机特征函数基类 3, 自定义其导函数的定义方式 (亦可使用自动微分), 修改所使用的激活函数而不必修改整体组装过程.

```python
class RFBase(ABC):
    # 抽象基类，表示随机特征(Random Features, RF)函数的接口

    @abstractmethod
    def forward(self, x):
        # 前向函数： 计算随机特征函数在点x 上的函数值
        pass

    @abstractmethod
    def first_derivative(self, x, axis):
        # 一阶导数： 计算随机特征函数在点x 上沿指定轴的偏导数
        pass

    @abstractmethod
    def second_derivative(self, x, axis1, axis2):
        # 二阶导数： 计算随机特征函数在点x 上沿指定轴1和轴2的二阶偏导数
        pass

    @abstractmethod
    def higher_order_derivative(self, x, order):
        # 高阶导数： 计算随机特征函数在点x 上的指定阶导数(order 为多重索引)
        pass
```

Listing 3 随机特征函数基类

## 6. 优化器

### 6.1. 线性最小二乘的优化

与传统深度学习模型相比, 随机特征方法 (RFM) 仅需优化输出层参数, 而内部层参数在随机初始化后保持不变. 这一特性将 RFM 的训练过程简化为一个线性最小二乘问题. 与随机梯度下降 (SGD) 等优化方法相比, 利用传统数值代数技术求解最小二乘问题可显著提升训练的精度与效率. 然而, 直接使用正规方程求解最小二乘问题可能引发数值不稳定的问题. 相比之下, 采用 QR 分解或 SVD 分解进行求解, 不仅能够提高精度, 还能增强稳定性. 其中, QR 分解具有更高的计算效率, 尤其适合并行计算; 而 SVD 分解则在数值稳定性方面表现更佳.

此外, 在参数规模较小或矩阵条件数较好的情况下, 也可以采用迭代法, 或者在经过预处理后使用该方法 [30]. 在使用迭代法时, 建议启用计算随机特征矩阵的use_sparse 选项. 启用该选项后, 软件将采用压缩稀疏行 (Compressed Sparse Row, CSR) 格式存储矩阵. 与其他稀疏格式相比, CSR 更适合迭代方法中频繁使用的矩阵乘法操作. 我们使用统一的模板实现了直接法和迭代法的求解器, 参考 Eigen 的设计, 构建了compute 与solve 两个接口. 前者负责进行矩阵分解, 后者则在分解结果的基础上支持多次高效求解.

### 6.2. 非线性最小二乘的优化

为解决非线性方程, 本文提出了一种通用的解决方案. 通过牛顿法对问题进行线性化处理后, 可利用线性最小二乘方法进行求解. 软件实现了四种基于牛顿法的改进算法, 包括线搜

索法、Levenberg-Marquardt 算法、信赖域反射算法以及 Dogleg 算法. 其中, 线搜索法因其算法简单且主要计算依赖于高度并行的 Householder 变换, 能够实现完全在 GPU 上的运行, 从而显著提升计算效率.

### 6.3. 逐批优化

随机梯度下降 (Stochastic Gradient Descent, SGD) 通过每次仅使用一小批样本来估计梯度, 从而显著降低了内存需求. 对于 RFM 模型, 在处理复杂情形, 尤其是高维问题时, 为了捕捉细节信息, 往往需要较多的采样点. 在这种情况下, 针对线性最小二乘问题或者非线性最小二乘问题中的每个线性步, 本文提出一种在不牺牲精度的前提下, 以时间换取空间, 进行训练的方法. 该方法将采样点划分为多个批次, 并利用 QR 分解的特性, 仅需占用只与特征数 $M$ 相关的内存空间. 该策略可以保证算法在各类场景下保持良好的稳定性和表现, 也可以启发动态调整采样策略的优化方法.

Givens 变换可以用于 QR 分解中消除矩阵的某个分量, 其表达式为:

$$
\begin{aligned}
G(i,k,\theta) &= I + s\left(e_i e_k^{\mathrm{T}} - e_k e_i^{\mathrm{T}}\right) + (c-1)\left(e_i e_i^{\mathrm{T}} + e_k e_k^{\mathrm{T}}\right) \\
&= \begin{bmatrix}
1 & & \vdots & & \vdots & & \\
& \ddots & \vdots & & \vdots & & \\
\cdots & \cdots & c & \cdots & s & \cdots & \cdots \\
& & \vdots & & \vdots & & \\
\cdots & \cdots & -s & \cdots & c & \cdots & \cdots \\
& & \vdots & & \vdots & \ddots & \\
& & \vdots & & \vdots & & 1
\end{bmatrix},
\end{aligned}
\tag{6.1}
$$

其中 $c = \cos\theta, s = \sin\theta$, $G(i,k,\theta)$ 是一个正交矩阵. 值得注意的是, Givens 变换仅影响两个特定的行. 假设我们使用 Givens 变换逐行消去矩阵中的元素, 那么在处理某一行时, 只会涉及到该行与至多前 $M$ 行的元素, 如图 2 所示. 因此, 考虑到内存限制, 我们可以将矩阵分为若干批次, 每次只处理一个批次的行, 并复用这些行的内存空间. 这样不仅避免了在内存中存储整个矩阵, 还实现了完整的 QR 分解. 如图 2 所示, 详细的计算流程见算法 6.1.

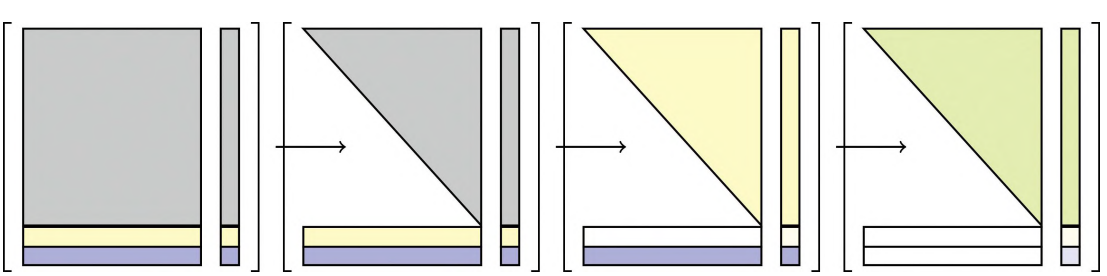


图 2 使用 Givens 变换进行 QR 分解的矩阵结构示意图

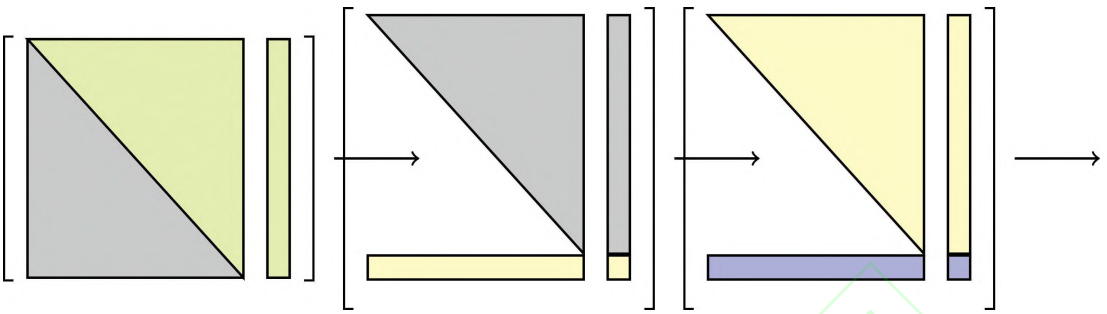


图 3 逐批 QR 分解矩阵结构示意图

**算法 6.1.** RFM 中的逐批 QR 分解

1: **输入:** 随机特征数 $M$, 每批条件数 *batch_size* , 总批次数 *n_batch*

2: **初始化:** 持续添加条件, 直到条件数超过 $M$, 形成 $A^{(0)}$ 和 $b^{(0)}$. 对 $A^{(0)}$ 做 QR 分解, 即 $A^{(0)} = Q^{(0)} \begin{pmatrix} R^{(0)} \\ \mathbf{0} \end{pmatrix}$, 保存 $R^{(0)} \in \mathbb{R}^{M\times M}$ 及 ${Q^{(0)}}^T b^{(0)}$ 的前 $M$ 个分量, 记为 $\hat{b}^{(0)}$

3: **for** $t = 1$ 到 *n_batch* **do**

4: **步骤 1:** 采样 *batch_size* 个条件, 得到 $\tilde{A}^{(t)}$ 和 $\tilde{b}^{(t)}$, 构造 $A^{(t)} = \begin{pmatrix} R^{(t-1)} \\ \tilde{A}^{(t)} \end{pmatrix}$ 及 $b^{(t)} = \begin{pmatrix} \hat{b}^{(t-1)} \\ \tilde{b}^{(t)} \end{pmatrix}$

5: **步骤 2:** 对 $\tilde{A}^{(t)}$ 做 QR 分解, 即 $A^{(t)} = Q^{(t)} \begin{pmatrix} R^{(t)} \\ \mathbf{0} \end{pmatrix}$

6: **步骤 3:** 计算 ${Q^{(t)}}^T b^{(t)}$, 取其前 $M$ 行记为 $\hat{b}^{(t)}$

7: **步骤 4:** 保存 $R^{(t)}$ 和 $\hat{b}^{(t)}$

8: **end for**

9: **输出:** 最终 QR 分解结果 $(R^{(n_batch)}, b^{(n_batch)})$, 解 $x$ 由上三角方程 $R^{(n_batch)}x = b^{(n_batch)}$ 给出

实际操作中, Householder 变换在并行计算中更为高效, 因此我们在软件中对一批采样点使用 Householder 变换代替 Givens 变换. 我们可以证明, 两者构造的 QR 分解是等价的, 即

**定理 6.1.** QR 分解的唯一性 设 $A \in \mathbb{R}^{N\times M} (N > M)$ 为满秩矩阵, 则存在唯一的 QR 分解:

$$A = QR \tag{6.2}$$

其中

- $Q \in \mathbb{R}^{N\times M}$ 为列正交矩阵 (即 $Q^TQ = I_n$),
- $R \in \mathbb{R}^{M\times M}$ 为主对角线元素为正的上三角矩阵.

**证明.** 假设存在两组 QR 分解:

$$A = Q_1 R_1 = Q_2 R_2, \tag{6.3}$$

其中 $Q_1, Q_2$ 均为列正交矩阵, $R_1, R_2$ 为主对角线元素为正的上三角矩阵. 令

$$U = R_2 R_1^{-1} = Q_2^T Q_1, \tag{6.4}$$

则 $U$ 既是正交矩阵又是上三角矩阵. 记 $u_k$ 为 $U$ 的第 $k$ 列. 归纳可得 $u_k = \pm e_k$, 其中 $e_k$ 为标准基向量. 由 $R_2 = UR_1$ 可知 $U_{kk} > 0$, 因为 $(R_1)_{kk}$ 和 $(R_2)_{kk}$ 均为正. 因此 $U = I$, 即 $Q_1 = Q_2$, $R_1 = R_2$.

## 7. 案　　例

本文所提供的软件旨在帮助随机特征方法的开发者与研究者节省实现相关算法的时间与精力. 该软件采用模块化设计, 用户可根据实际需求灵活选取或调整相应模块, 而无需从零开始编写完整程序. 各模块内部亦针对随机特征方法的特点进行了效率优化. 在使用该软件求解不同类型的方程时, 用户只需更改所调用的微分操作与矩阵组装方式, 即可完成适配. 下文将通过若干典型案例对这一过程进行示范说明.

值得一提的是, 在二维场景中, 该软件能够在任意分辨率下准确表达每一个像素; 而在三维场景中, 我们基于有符号距离函数 (Signed Distance Function, SDF), 采用光线步进 (Ray Marching) 技术进行渲染. 渲染得到的图像将借助 Python 原生绘图库 Matplotlib 进行标注与后续处理. 文中所记录的运行时间, 均指在 RTX 4060 显卡上完成一个完整案例 (包括采样、组装与求解) 的总耗时.

### 7.1. Helmholtz 方程

考虑定义在区间 $\Omega = [0, 8]$ 上的一维 Helmholtz 方程, 施加 Dirichlet 边界条件:

$$\begin{cases} \dfrac{\mathrm{d}^2 u(x)}{\mathrm{d}x^2} + \lambda u(x) = f(x), \quad x \in \Omega, \\ u(0) = c_1, \quad u(8) = c_2, \end{cases}$$

其中波数 $\lambda = 4$. 选取精确解为

$$u(x) = \sin\left(3\pi x + \tfrac{3\pi}{20}\right) \cos\left(2\pi x + \tfrac{\pi}{10}\right) + 2.$$

相应地, $c_1, c_2, f$ 可由解析计算获得.

使用 pyRFM 可以便利地进行求解实现. 首先, 定义一维区域并初始化 RFM 模型:

```
domain = pyrfm.Line1D(x1=0.0, x2=8.0)
model = pyrfm.RFMBase(
    dim=1, n_hidden=50, domain=domain,
    n_subdomains=args.M / 50, pou=pyrfm.PsiA
)
```

其中:

- Line1D 定义了区间 $[0, 8]$;
- RFMBase 创建随机特征模型实例, n_hidden 为隐藏层特征数;
- n_subdomains 指定子域数;
- pou 为单位分解函数 (partition of unity) 类型.

接下来, 在区域内部与边界采样:

```
x_in = domain.in_sample(args.Q, with_boundary=False)
x_on = domain.on_sample(2)
```

这里:

- x_in 为内部采样点 (不含边界);
- x_on 为边界采样点 (两个端点).

然后, 基于采样点计算特征矩阵及其二阶导函数特征矩阵, 并构造界面条件:

```
A_in = model.features(x_in).cat(dim=1)
A_in_xx = model.features_second_derivative(
    x_in, axis1=0, axis2=0).cat(dim=1)
A_on = model.features(x_on).cat(dim=1)
A_interface = model.add_c_condition((args.M / 50 - 1)).cat(dim=1)
```

其中:

- A_in 为特征矩阵 $\Phi(x_i)$;
- A_in_xx 为二阶导 $\partial_{xx}\Phi(x_i)$;
- A_on 为边界点的特征矩阵;
- A_interface 为子域界面的连续性约束, 当采样具备更强光滑性的单位分解函数 pyrfm.PsiB 时, 则不需要额外添加该约束.

接着, 将 PDE 主体、边界条件与界面条件拼接为整体算子矩阵 $A$:

```
A = pyrfm.concat_blocks([
    [A_in_xx - Lambda * A_in],
    [A_on],
    [A_interface]
])
```

对应的右端项 $f$ 由内部源项 $f(x)$、边界条件 $g(x)$ 和界面零约束组成:

```
f_in = func_f(x_in).view(-1, 1)
f_on = func_g(x_on).view(-1, 1)

f = pyrfm.concat_blocks([
    [f_in],
    [f_on],
    [torch.zeros(A_interface.shape[0], 1)]
])
```

最后, 调用求解器即可获得数值解:

```
model.compute(A).solve(f)
```

不同参数下的结果整理如下:

**表 1 不同参数下的 Helmholtz 方程求解结果**

| 配点数 $N$ | 特征数 $M$ | $\Psi$ 类型 | 最小二乘残差 | 相对误差 | 时间 (秒) |
|---|---|---|---|---|---|
| 208 | 200 | A | $1.2711 \times 10^{-5}$ | $7.7766 \times 10^{-4}$ | 0.90 |
| 416 | 400 | A | $7.0842 \times 10^{-10}$ | $1.6094 \times 10^{-7}$ | 0.07 |
| 832 | 800 | A | $3.8472 \times 10^{-12}$ | $8.5486 \times 10^{-10}$ | 0.24 |
| 1664 | 1600 | A | $2.1508 \times 10^{-8}$ | $4.1787 \times 10^{-7}$ | 0.84 |
| 202 | 200 | B | $6.9784 \times 10^{-4}$ | $1.9711 \times 10^{-1}$ | 0.02 |
| 402 | 400 | B | $2.2708 \times 10^{-8}$ | $1.0734 \times 10^{-5}$ | 0.04 |
| 802 | 800 | B | $1.2516 \times 10^{-10}$ | $4.9078 \times 10^{-8}$ | 0.11 |
| 1602 | 1600 | B | $1.6365 \times 10^{-11}$ | $2.3350 \times 10^{-9}$ | 0.44 |

注: $M$ 为总特征数, 每 50 个特征对应一个子区域; $\Psi$ 类型为 A 表示使用单位分解函数 2.4, 为 B 表示单位分解函数 2.5.

### 7.2. 多尺度随机特征下的泊松方程

通过在全域上引入一组随机特征, 可以构造多尺度基函数. 考虑定义在区域 $\Omega = [0,1] \times [0,1]$ 上, 带有 Dirichlet 边界条件的 Poisson 方程:

$$\Delta u(x,y) = f(x,y), \quad (x,y) \in \Omega.$$

边界条件为:

$$u(x,0) = g_1(x), \quad u(x,1) = g_2(x),$$
$$u(0,y) = h_1(y), \quad u(1,y) = h_2(y).$$

我们选取一个具有混合频率的精确解:

$$\begin{aligned} u(x,y) = &-\frac{1}{2}\left[\frac{3}{2}\cos\left(\pi x + \frac{2\pi}{5}\right) + 2\cos\left(2\pi x - \frac{\pi}{5}\right)\right]\left[\frac{3}{2}\cos\left(\pi y + \frac{2\pi}{5}\right) + 2\cos\left(2\pi y - \frac{\pi}{5}\right)\right] \\ &-\frac{1}{2}\left[\frac{3}{2}\cos\left(2\pi x + \frac{4\pi}{5}\right) + 2\cos\left(4\pi x - \frac{2\pi}{5}\right)\right]\left[\frac{3}{2}\cos\left(2\pi y + \frac{4\pi}{5}\right) + 2\cos\left(4\pi y - \frac{2\pi}{5}\right)\right]. \end{aligned}$$

由此可直接计算得到函数 $g_1, g_2, h_1, h_2$ 以及源项 $f$. 在 2.6 的基础上, 我们增加一组定义定义整个区域 $\Omega$ 基函数

$$\phi_{g,j}(\boldsymbol{x}) := \sigma\left(\sum_{k=1}^{d} a_{k,j}\frac{x_k - 0.5}{0.5} + b_j\right), \quad j = 1, \ldots, M_g$$

其中 $\boldsymbol{x} = (x_k) \in \mathbb{R}^d$. 此时, 最终的近似解为:

$$u_M(\boldsymbol{x}) = \sum_{n=1}^{M_p} \psi_n(\boldsymbol{x}) \sum_{j=1}^{M_n} u_{nj}\phi_{nj}(\boldsymbol{x}) + \sum_{j=1}^{M_g} u_{gj}\phi_{gj}(\boldsymbol{x}) \tag{7.1}$$

对原模型进行继承与修改, 可以定义多尺度的随机特征模型, 例如

```python
class MultiScaleRFM(pyrfm.RFMBase):
    def __init__(self, dim, n_hidden, domain, n_subdomains, pou = pyrfm.PsiB,
                 rf=pyrfm.RFTanH, *args, **kwargs):
        super().__init__(dim = dim,
                         n_hidden = n_hidden,
                         domain = domain,
                         n_subdomains = n_subdomains,
                         pou = pou,
                         rf= rf,
                         *args, **kwargs)
        bounding_box = self.domain.get_bounding_box()
        center = torch.tensor([(bounding_box[2*i+1] + bounding_box[2*i]) / 2.0
            for i in range(dim)], dtype=self.dtype, device=self.device)
        radius = torch.tensor([(bounding_box[2*i+1] - bounding_box[2*i]) / 2.0
            for i in range(dim)], dtype=self.dtype, device=self.device)
        self.global_model = rf(dim = dim, center = center, radius = radius,
            n_hidden = n_hidden)

    def features(self, x: torch.Tensor, use_sparse: bool = False) -> Tensor:
        return torch.cat([super().features(x, use_sparse).cat(dim = 1), self.
            global_model.forward(x)], dim = 1)

    def features_derivative(self, x: torch.Tensor, axis: int, use_sparse: bool =
         False) -> Tensor:
        return torch.cat([super().features_derivative(x, axis, use_sparse).cat(
            dim = 1), self.global_model.first_derivative(x, axis)], dim = 1)

    def features_second_derivative(self, x: torch.Tensor, axis1: int, axis2: int
        , use_sparse: bool = False) -> Tensor:
        return torch.cat([super().features_second_derivative(x, axis1, axis2,
            use_sparse).cat(dim = 1), self.global_model.second_derivative(x,
            axis1, axis2)], dim = 1)
```

求解该问题的结果如表 2 所示. 此外, 我们对随机特征方法与传统有限元方法的性能进行了比较. 实验中, pyRFM 框架在 GPU 上进行求解, 而有限元方法实现 FEniCS 实现则通过 MPI 并行化提升效率. 比较结果见表 3. 结果表明, 在达到相同精度的条件下, 有限元方法需要更高的自由度, 因此随机特征方法在该类问题上表现出更优的计算效率.

**表 2 Poisson 方程在混合频情况下的求解结果**

| 配点数 | 特征数 $M$ | Multiscale | 最小二乘残差 | 相对误差 | 时间 (秒) |
|---|---|---|---|---|---|
| 1920 | 1200 | False | $6.2786 \times 10^{-11}$ | $1.6889 \times 10^{-7}$ | 0.09 |
| 4320 | 2700 | False | $3.6746 \times 10^{-12}$ | $9.4586 \times 10^{-9}$ | 0.49 |
| 7680 | 4800 | False | $1.0481 \times 10^{-12}$ | $4.8534 \times 10^{-9}$ | 1.95 |
| 1920 | 1500 | True | $1.2506 \times 10^{-11}$ | $3.2116 \times 10^{-8}$ | 0.10 |
| 4320 | 3000 | True | $1.3591 \times 10^{-12}$ | $1.9391 \times 10^{-9}$ | 0.55 |
| 7680 | 5100 | True | $5.8693 \times 10^{-13}$ | $3.3708 \times 10^{-9}$ | 2.12 |

注: 配点数表示离散点的数量; $M$ 为总特征数 (每 50 个特征对应一个子区域); Multiscale 表示是否采用多尺度随机特征.

**表 3 RFM 与 FEM 的 Poisson 方程求解结果对比**

| 方法 | 配点数/节点数 | 特征数/单元数 | 线性系统残差 | 相对误差 | 时间 (秒) |
|---|---|---|---|---|---|
| RFM | 1920 | 1200 | $6.2786 \times 10^{-11}$ | $1.6889 \times 10^{-7}$ | 0.09 |
| RFM | 4320 | 2700 | $3.6746 \times 10^{-12}$ | $9.4586 \times 10^{-9}$ | 0.49 |
| RFM | 7680 | 4800 | $1.0481 \times 10^{-12}$ | $4.8534 \times 10^{-9}$ | 1.95 |
| FEM | 40000 | 40401 | $6.6994 \times 10^{-13}$ | $1.8424 \times 10^{-4}$ | 0.45 |
| FEM | 160000 | 160801 | $1.3989 \times 10^{-12}$ | $4.6061 \times 10^{-5}$ | 2.02 |
| FEM | 360000 | 361201 | $2.2379 \times 10^{-12}$ | $2.0472 \times 10^{-5}$ | 6.28 |

### 7.3. 非线性 Poisson 方程

考虑 $\Omega = [0,1] \times [0,1]$ 上的包含非线性项的 Poisson 方程,

$$\begin{cases} -\Delta u(x,y) + u^2(x,y) = f(x,y), \quad (x,y) \in \Omega, \\ u(x,0) = g_1(x), \quad u(x,1) = g_2(x), \\ u(0,y) = h_1(y), \quad u(1,y) = h_2(y). \end{cases} \tag{7.2}$$

然后我们取 $u(x,y)$ 如下,

$$u(x,y) = -\left[\frac{3}{2}\cos\left(\pi x + \frac{2\pi}{5}\right) + 2\cos\left(2\pi x - \frac{\pi}{5}\right)\right]\left[\frac{3}{2}\cos\left(\pi y + \frac{2\pi}{5}\right) + 2\cos\left(2\pi y - \frac{\pi}{5}\right)\right]. \tag{7.3}$$

带入上式可求得 $f(x), g_1(x), g_2(x), h_1(y), h_2(y)$.

我们采用第 6.2 节中介绍的四种非线性最小二乘求解器对该方程进行求解, 计算结果如表 4 所示. 其中, 线搜索方法基于 PyTorch 的原生实现, 而其余三种方法则调用了 SciPy 提供的相应求解器. 在 GPU 计算方面, 线搜索方法的整个求解过程均在 GPU 上完成; 而对于其余三种方法, 仅雅可比矩阵的计算和残差项的评估在 GPU 上执行, 其余步骤仍在 CPU 上进行 (数据在内存与显存之间的传输可能导致性能损失). 线搜索方法为牛顿法的一种简单改进形式, 在当前问题中表现出较好的效果. 然而, 对于更复杂的非线性问题, 仍可能需要更精细的非线性处理策略, 以确保算法的稳健性与收敛性.

**表 4 不同非线性最小二乘求解方法在 CPU 与 GPU 上得到的解的误差与耗时对比**

| 问题规模 | 方法 | $L^2$ 误差 | | 时间 (s) | |
|---|---|---|---|---|---|
| | | CPU | GPU | CPU | GPU |
| $9921 \times 1200$ | Newton(line search) | $1.8460 \times 10^{-8}$ | $3.8080 \times 10^{-8}$ | 32.40 | 3.32 |
| | Trust Region Reflective (TRF) | $2.0270 \times 10^{-8}$ | $1.8860 \times 10^{-8}$ | 39.64 | 37.22 |
| | Levenberg–Marquardt (LM) | $1.8360 \times 10^{-8}$ | $2.0520 \times 10^{-8}$ | 179.06 | 197.93 |
| | Dogbox | $2.6280 \times 10^{-7}$ | $6.9530 \times 10^{-7}$ | 30.64 | 29.93 |

### 7.4. 二维复杂几何中的 Stokes 流动

我们考虑如下所定义的 Stokes 流动问题:

$$\begin{cases} -\Delta u(x) + \nabla p(x) = f(x), & x \in \Omega, \\ \nabla \cdot u(x) = 0, & x \in \Omega, \\ u(x) = U(x), & x \in \partial\Omega. \end{cases}$$

在该问题中, 压力函数 $p$ 仅能确定到一个常数项. 不失一般性, 我们在左下角固定 $p$ 的取值.

首先, 我们考虑带有显式解和非齐次边界条件的情形. 此时区域 $\Omega$ 为方形 $(0,1) \times (0,1)$, 其中挖有三个半径为 0.1 的圆孔, 圆心分别位于 $(0.5, 0.2)$、$(0.2, 0.8)$、$(0.8, 0.8)$. 精确解定义如下:

$$\begin{aligned} u &= x + x^2 - 2xy + x^3 - 3xy^2 + x^2y, \\ v &= -y - 2xy + y^2 - 3x^2y + y^3 - xy^2, \\ p &= xy + x + y + x^3y^2 - \tfrac{4}{3}. \end{aligned}$$

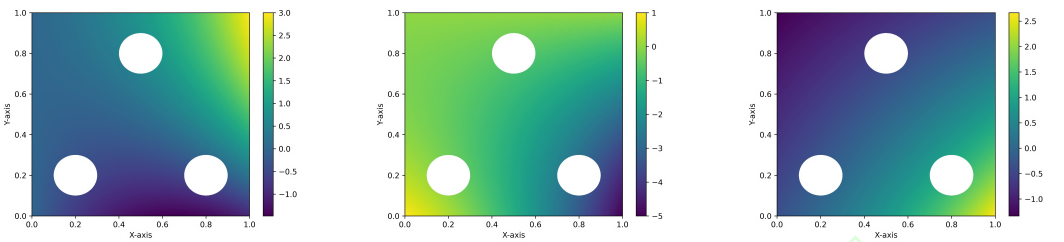
为了对结果进行精确的可视化呈现, 我们分别为 2D 与 3D 提供了可视化模块. 该模块将对结果进行像素级的准确呈现, 并且对于非规则区域, 将利用 SDF 对区域进行准确的可视化. 例如对于求解后的该模型, 可以使用

```
visualizer = pyrfm.RFMVisualizer2D(model, component_idx=0)
visualizer.plot()
visualizer.show()
visualizer.savefig('stokes_flow.png', dpi=600)
```

其中:

- `visualizer` 为创建的可视化实例;
- `component_idx` 指定了进行可视化的分量;

`RFMVisualizer2D` 沿用了常用绘图库 Matplotlib 的方法, 以满足各种定制化的需求. 其可视化的结果如图 4 所示.




(a) $u$ (b) $v$ (c) $p$

图 4 Stokes 流动可视化结果

### 7.5. 复杂几何结构下的三维线弹性方程

考虑一个具有复杂几何结构的三维弹性问题, 其数学模型为:

$$\begin{cases} -\nabla \cdot \boldsymbol{\sigma}(\mathbf{u}(\mathbf{x})) = \mathbf{B}(\mathbf{x}), & \mathbf{x} \in \Omega, \\ \boldsymbol{\sigma}(\mathbf{u}(\mathbf{x})) \cdot \mathbf{n} = \mathbf{N}(\mathbf{x}), & \mathbf{x} \in \Gamma_N, \\ \mathbf{u}(\mathbf{x}) = \mathbf{U}(\mathbf{x}), & \mathbf{x} \in \Gamma_D, \end{cases}$$

其中 $\mathbf{u} : \Omega \to \mathbb{R}^2$ 表示位移场, $\boldsymbol{\sigma}$ 为由 $\mathbf{u}$ 导出的应力张量, $\mathbf{B}$ 为区域 $\Omega$ 内的体力密度, $\mathbf{N}$ 为边界 $\Gamma_N$ 上的表面力, $\mathbf{U}$ 为边界 $\Gamma_D$ 上的位移边界条件. 边界分解满足 $\partial\Omega = \Gamma_N \cup \Gamma_D$ 且 $\Gamma_N \cap \Gamma_D = \emptyset$.

区域 $\Omega$ 由二维截面沿 $x$ 方向拉伸而成. 几何包含 “窄缝” 结构, 这给网格生成带来了显著困难. 二维截面定义为:

$$\Omega_{2\mathrm{D}} = \big(\Omega_1 \cap \Omega_2\big) \setminus \Omega_3,$$

其中:

- $\Omega_1$ 为基准方形:
$$\Omega_1 = \{(y, z) \mid 1.5 \le y \le 2.5,\ 1.0 \le z \le 2.0\},$$
- $\Omega_2 = C_1 \cup C_2 \cup C_3$ 为三保留圆的并集,
- $\Omega_3 = D_1 \cup D_2 \cup D_3 \cup D_4$ 为四切除圆的并集.

各圆的参数如下表所示:

| 符号 | 圆心$(y, z)$ | 半径 |
|---|---|---|
| $C_1$ | $(1.9576,\ 1.2457)$ | 0.0502 |
| $C_2$ | $(2.1249,\ 1.5534)$ | 0.2743 |
| $C_3$ | $(1.9883,\ 1.5191)$ | 0.2666 |
| $D_1$ | $(2.1955,\ 1.7094)$ | 0.1413 |
| $D_2$ | $(2.0466,\ 1.0623)$ | 0.1596 |
| $D_3$ | $(1.7483,\ 1.4981)$ | 0.1915 |
| $D_4$ | $(1.9530,\ 1.6900)$ | 0.0920 |

我们固定其底面, 并在顶面施加 $y$ 方向 0.1 mm 的位移, 观察其 Von Mises 应力分布. 材料参数取: 杨氏模量 $E = 3 \times 10^7$ Pa, 泊松比 $\nu = 0.3$. 三维的随机特征方法的可视化在指定的视角下, 通过计算每个像素对应的光线与几何的交点, 可视化物理场在几何表面的分布. SDF 的定义适用于使用光线步进来计算交点. 这种渲染方式不依赖于传统表面网格的生成, 对复杂几何具有良好的适应性. 对该案例中的几何体, 可使用 3D 渲染模块RFMVisualizer3D 进行渲染. 对于特殊定义的物理场, 例如 von Mises 应力, 可以继承并修改物理场的计算方式, 如下所示:

```python
class CustomVisulizer(pyrfm.RFMVisualizer3D):
    def __init__(self, model: pyrfm.RFMBase, t=0.0, resolution=(1920, 1080),
        component_idx=0, view='iso', ref=None):
        super().__init__(model, t, resolution, component_idx, view, ref)

    def compute_field_values(self, pts_hit, hits):
        ...
        # von Mises
        vonmises = torch.sqrt(
            0.5 * ((sxx - syy) ** 2 + (syy - szz) ** 2 + (szz - sxx) ** 2) + 3 *
                (sxy ** 2 + sxz ** 2 + syz ** 2)
        )
        return vonmises.detach().cpu().numpy()
```

最终可得到 Von Mises 应力的三维分布, 如图 5 所示.

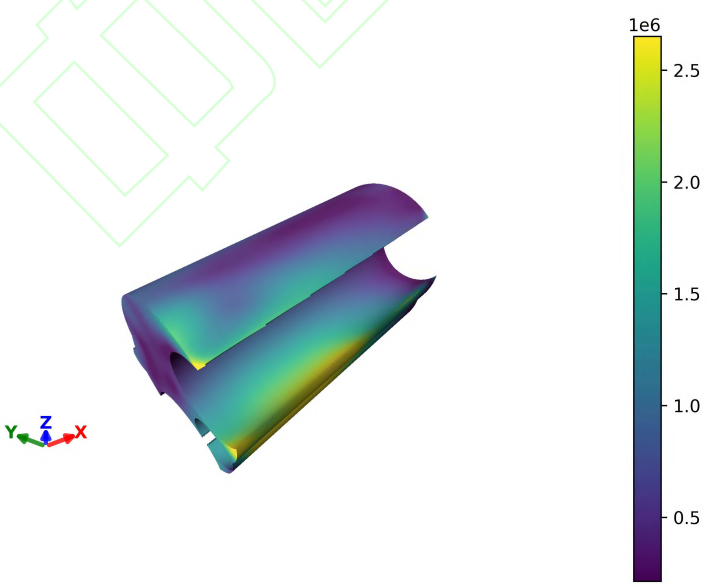



图 5 Von Mises 应力分布

## 8. 结 论

本文介绍的软件 pyRFM 基于随机特征方法, 为偏微分方程的高效求解提供了一种全新的实现路径. 与传统数值方法相比, 它在处理复杂几何时展现出更强的灵活性与适应性; 同时, 相较于深度学习框架下的物理信息神经网络等方法, 它在精度与计算效率上具有明显优势. 本文着重介绍了软件的设计思路与提高实现性能的要求, 并以多个案例阐释软件的易用性与可拓展性.

该软件旨在为科研人员与工程实践提供一个高效且相对易用的计算框架, 希望能在一定程度上推动随机特征方法在科学计算中的应用与探索. 当前版本在性能与扩展性方面已初步展现出一定潜力, 但仍有诸多值得改进和完善之处. 未来, 我们将继续在功能扩展、算法优化以及跨领域适配等方面努力探索.

## 参 考 文 献

[1] LeVeque R J. Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems[M]. SIAM, 2007.

[2] Zienkiewicz O C, Taylor R L, Zhu J Z. The finite element method: its basis and fundamentals[M]. Elsevier, 2005.

[3] Shen J, Tang T, Wang L L. Spectral methods: algorithms, analysis and applications[M]. volume 41. Springer Science & Business Media, 2011.

[4] E W, Han J, Jentzen A. Deep Learning-Based Numerical Methods for High-Dimensional Parabolic Partial Differential Equations and Backward Stochastic Differential Equations[J]. Commun. Math. Stat., 2017, 5(4): 349–380.

[5] Han J, Jentzen A, E W. Solving high-dimensional partial differential equations using deep learning[J]. Proc. Natl. Acad. Sci. U.S.A., 2018, 115(34): 8505–8510.

[6] E W, Yu B. The Deep Ritz Method: A Deep Learning-Based Numerical Algorithm for Solving Variational Problems[J]. Commun. Math. Stat., Mar 2018, 6(1): 1–12.

[7] Sirignano J A, Spiliopoulos K. DGM: A deep learning algorithm for solving partial differential equations[J]. J. Comput. Phys., 2018, 375: 1339–1364.

[8] Raissi M, Perdikaris P, Karniadakis G E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations[J]. J. Comput. Phys., 2019, 378: 686–707.

[9] Zang Y, Bao G, Ye X, Zhou H. Weak Adversarial Networks for High-dimensional Partial Differential Equations[J]. J. Comput. Phys., 2020, 411: 109409.

[10] Lu L, Jin P, Pang G, Zhang Z, Karniadakis G E. Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators[J]. Nature Machine Intelligence, March 2021, 3(3): 218–229.

[11] Li Z, Kovachki N B, Azizzadenesheli K, Liu B, Bhattacharya K, Stuart A, Anandkumar A. Fourier Neural Operator for Parametric Partial Differential Equations[C]. In International Conference on Learning Representations, October 2020.

[12] Huang G B, Zhu Q Y, Siew C K. Extreme Learning Machine: Theory and Applications[J]. Neurocomputing, December 2006, 70(1–3): 489–501.

[13] Neal R M. Bayesian Learning for Neural Networks[D]. PhD thesis, University of Toronto, 1995.

[14] Rahimi A, Recht B. Random Features for Large-Scale Kernel Machines[C]. In Advances in Neural Information Processing Systems, volume 20. Curran Associates, Inc., 2007.

[15] Batlle P, Chen Y, Hosseini B, Owhadi H, Stuart A M. Error Analysis of Kernel/GP Methods for Nonlinear and Parametric PDEs[J]. Journal of Computational Physics, January 2025, 520: 113488.

[16] E W, Ma C, Wu L. The Barron Space and the Flow-Induced Function Spaces for Neural Network Models[J]. Constructive Approximation, February 2022, 55(1): 369–406.

[17] Ming P, Yu H. Spectral Connvergece of Random Feature Method in One Dimension[M]. July 2025.

[18] Li Z, Ton J F, Oglic D, Sejdinovic D. Towards a Unified Analysis of Random Fourier Features[J]. Journal of Machine Learning Research, 2021, 22(108): 1–51.

[19] Yang Y, Hou M, Luo J. A novel improved extreme learning machine algorithm in solving ordinary differential equations by Legendre neural network methods[J]. Advances in Difference Equations, 2018, 2018(1): 1–24.

[20] Calabrò F, Fabiani G, Siettos C. Extreme learning machine collocation for the numerical solution of elliptic PDEs with sharp gradients[J]. Computer Methods in Applied Mechanics and Engineering, 2021, 387(1): 114–188.

[21] Dong S, Li Z. Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations[J]. Computer Methods in Applied Mechanics and Engineering, 2021, 387(1): 114–129.

[22] Chen J, Chi X, E W, Yang Z. Bridging Traditional and Machine Learning-Based Algorithms for Solving PDEs: The Random Feature Method[J]. J. Mach. Learn., 2022, 1(3): 268–298.

[23] Zhang Z, Bao F, Ju L, Zhang G. Transferable Neural Networks for Partial Differential Equations[J]. Journal of Scientific Computing, April 2024, 99(1): 2.

[24] Shang Y, Wang F, Sun J. Randomized Neural Network with Petrov–Galerkin Methods for Solving Linear and Nonlinear Partial Differential Equations[J]. Communications in Nonlinear Science and Numerical Simulation, December 2023, 127: 107518.

[25] Chi X, Chen J, Yang Z. The random feature method for solving interface problems[J]. Computer Methods in Applied Mechanics and Engineering, February 2024, 420: 116719.

[26] Sun Y, Chen J. Two-level random feature methods for elliptic partial differential equations over complex domains[J]. Computer Methods in Applied Mechanics and Engineering, 2025, 441: 117961.

[27] Chen J, E W, Sun Y. Optimization of Random Feature Method in the High-Precision Regime[J]. Communications on Applied Mathematics and Computation, March 2024.

[28] Chen J, E W, Luo Y. The random feature method for time-dependent problems[J]. East Asian Journal on Applied Mathematics, 2023, 13(3): 435–463.

[29] Park J J, Florence P, Straub J, Newcombe R, Lovegrove S. DeepSDF: Learning Continuous Signed Distance Functions for Shape Representation[C]. In 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), June 2019, 165–174.

[30] Chen J, Tan L. High-precision randomized iterative methods for the random feature method[M]. 2024.